

```
In [ ]: 1 # Zaimportować niezbędne biblioteki do wykonania poniższych zadań
        2
```

Zadanie 1

Utworzyć slidery do zmiany koloru tła

1. Zaimportować odpowiednie biblioteki dla serwisu /clear
2. Utworzyć funkcję `set_color` przyjmującą 3 argumenty koloru, która ustawia odpowiednie wartości parametrów koloru tła.
3. Wyczyścić serwisem /clear tło mapy, aby możliwa była aktualizacja koloru.
4. Utworzyć widget z 3 sliderami ustawiającymi kolor.

```
In [ ]: 1
```

Zadanie 2

Utworzyć publisher'a i slidery do wysyłania prędkości postępowej i obrotowej do robota. Można skorzystać z dostępnych sliderów na początku tego skryptu. Wiadomość o prędkościach powinna pojawiać się na topicu `/nazwa_robota/cmd_vel/slider`.

```
In [ ]: 1
```

Zadanie 3

Utworzyć dwie kontrolki **FloatText** do wprowadzania prędkości i przycisk. Wyświetlić wartości wpisanych wartości po kliknięciu przycisku

```
In [ ]: 1
```

Zadanie 4

Rozbudować program z zadania 3-ego o wysyłanie prędkości do ROS na topic `/nazwa_robota/cmd_vel/text`.

```
In [ ]: 1
```

Zadanie 5

Utworzyć checkbox'a do określania czy wykorzystywane są prędkości ze sliderów czy z pól tekstowych. Po ustawieniu wartości po kliknięciu przycisku należy ustawić parametr **slider_controler** zgodnie z aktualną wartością.

```
In [ ]: 1
```

Zadanie 6

Warunkiem do wykonania zadania jest prawidłowe wykonanie zadań 2,3,4,5. Odczytać wartość z parametru **slider_controler** z ROS, który informuje o źródle sygnału z prędkości czy to są slidery czy pola tekstowe. Dla wartości True wartości sterujące powinny być przekazywane ze sliderów, a dla False z kontroltek tekstowych. Uzupełnić poniższą klasę.

```
In [ ]: 1 rospy.set_param(..., ...) # UZUPEŁNIĆ ustawić parametr velocity_source,
        2 # 1 - źródło slidery, 2 - float_text
        3 class VelocityController:
        4     def __init__(self):
        5         # UZUPEŁNIĆ subskryberem od topicu ze sliderów, do odczytu
        6         # wykorzystać metodę klasy callback_slider. Odwołanie się do metod
        7         # klasy następuje poprzez
        8         # self.nazwa_metody w klasie.
        9         slider_sub = ...
       10
       11     # UZUPEŁNIĆ subskryberem od wartości prędkości wprowadzanych z
       12     # pól tekstowych
```

```

13         #callback_float_text
14         float_text_sub = ...
15
16         # UZUPEŁNIĆ utworzyć publisher, który będzie wysyłał wiadomości w
17         # zależności od ustawionego parametru velocity_source do robota.
18         # Gdzie wartość parametru 1 oznacza, że źródłem jest slider,
19         # a wartość 2 - float_text. Dla pozostałych wartości parametru
20         # robot stoi w miejscu.
21         # Wysyłana jest prędkość 0 zatrzymująca robota. Wiadomość powinna
22         # zostać opublikowana na topic sterującym robotem /nazwa_robota/cmd_vel
23         self.vel_pub = ...
24
25     def callback_slider(self, msg_data):
26         # UZUPEŁNIĆ zweryfikować stan parametru i jeśli tryb jest właściwy
27         # przekazywać prędkości sterujące
28         ...
29
30     def callback_float_text(self, msg_data):
31         # UZUPEŁNIĆ zweryfikować stan parametru i jeśli tryb jest właściwy
32         # przekazywać prędkości sterujące
33         ...
34

```

```

In [ ]: 1 VelocityControler()
        2

```